

Custom attribute approval gate

Stopping custom student attributes from multiplying — an AI-checked, second-person-approved creation gate.

The problem

Admins can create custom student attributes to capture data a statutory return needs but the core model doesn't hold. They are stored as rows (EAV: `student_custom_field` + `student_custom_value`), so the schema never changes. But every attribute widens the statutory export's join, and today anyone with access can create one directly. Nothing stops duplicates, fields the core model already holds, or one-off attributes nobody maps. Left alone, a tenant drifts towards hundreds of noisy attributes.

The idea

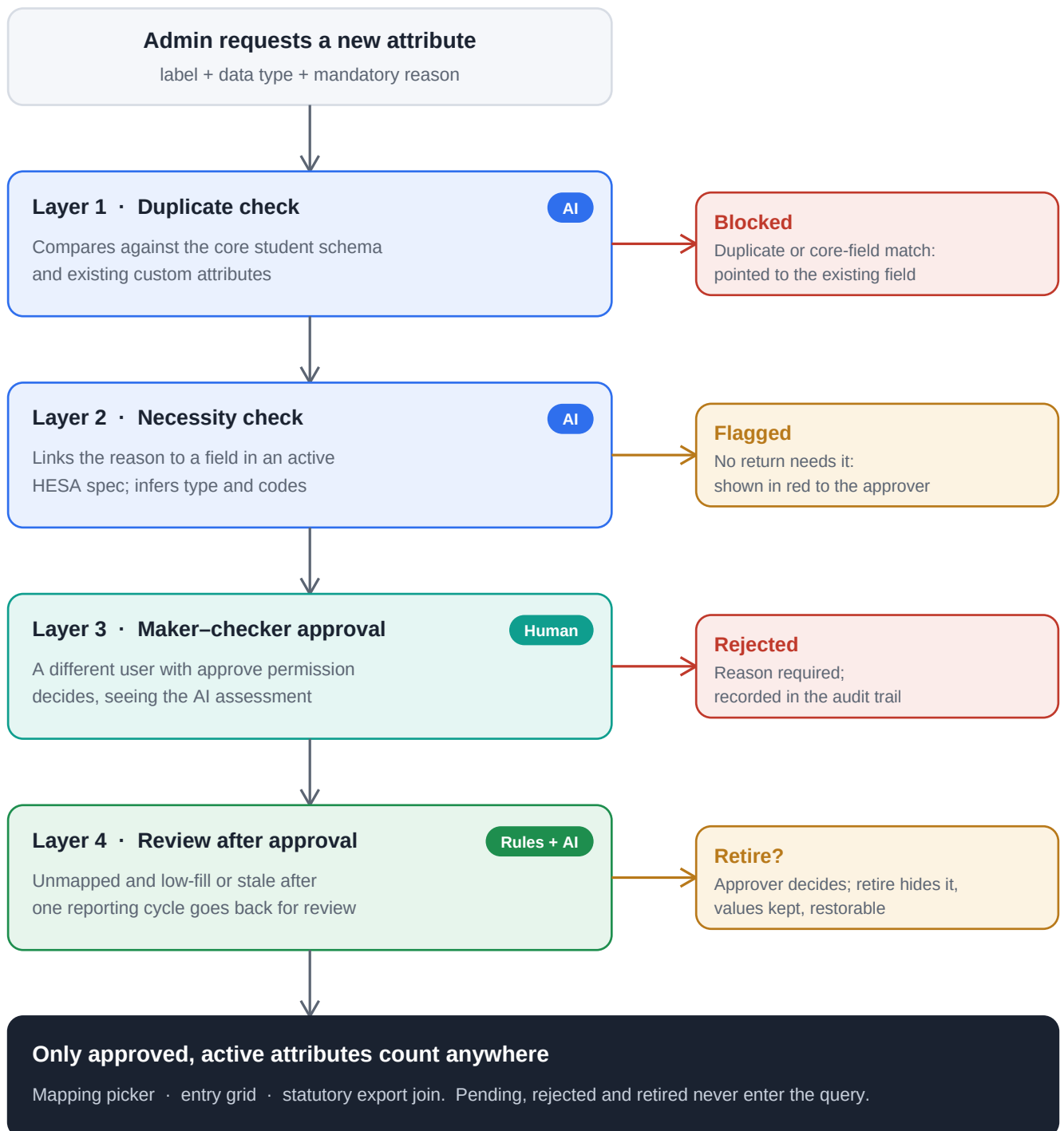
Nobody creates an attribute directly any more. They raise a **request**. It becomes real only after an **AI check** and a **second person's approval**, and it is **reviewed** afterwards so unused ones don't pile up.

Layer	Who	What it stops
1 Duplicate check	AI	Re-creating an existing custom attribute or a field the core model already holds
2 Necessity check	AI	Attributes no active statutory return actually needs
3 Maker-checker approval	Human	One person creating attributes unilaterally
4 Review after approval	Rules + AI	Approved attributes that later fall out of use

Why this solves the join problem

Only **approved, active** attributes count anywhere: the mapping picker, the entry grid and the statutory export's join. Pending, rejected and retired attributes never enter the query. The number that matters is only ever the vetted set — a few dozen, not a thousand.

The gate, end to end



The four layers in detail

Layer 1 — Duplicate check (AI)

When someone requests “Care leaver status”, the AI compares it with the core student schema and the existing custom attributes.

- Matches an existing custom attribute → **blocked**: “care_leaver_flag already exists.”
- Matches a core field → **blocked**: “The system already holds this as student.x; map to that.”
- Near match → **allowed through**, but the approver sees the warning.

Layer 2 — Necessity check (AI)

An attribute should exist because a statutory return needs it. The AI reads the stated reason against the active HESA spec packs and links the request to a specific field, e.g. “needed for SEXORT in HESA_STUDENT 2026/27.”

- No link found → marked “**Not required by any active return**”, shown in red to the approver.
- Link found → also suggests the **data type and allowed codes**, so the attribute starts out properly typed.

Layer 3 — Maker-checker approval (human)

The request sits in **pending**. A different user with an approve permission reviews it, seeing the AI assessment (duplicate risk, spec link, recommendation) next to the reason.

- The requester can **never approve their own** request.
- Rejections need a reason; every decision is recorded in the audit trail.

Layer 4 — Review after approval (rules + AI)

Approved isn't permanent. Attributes that fall out of use go back to the approver as “retire this?”. How “in use” is decided is set out on the next page.

How “not in use” is decided

Not by AI judgement. “In use” is defined by facts the system can measure; the AI only explains the result to the approver.

Signal	How it's measured	What it tells us
Mapped to a return	Any <code>ReportFieldMapping</code> in a non-superseded profile has <code>custom.<key></code> as its source	Strongest signal. If a live return reads it, it's in use by definition.
Fill rate	Students with a value ÷ active students	Whether anyone is actually entering data
Freshness	Latest <code>updated_at</code> on its values	Whether the data is being maintained
Spec still requires it	The HESA field linked at approval still exists in the current spec version	Whether its original reason still holds

The rule

- **Never flagged:** anything mapped in an active return profile, whatever the fill rate.
- **Flagged for review** only if it is **not mapped anywhere** and either the fill rate is low (e.g. under 5%), there have been no value updates for a full reporting cycle, or the spec field it was created for no longer exists.

Two safeguards

- **The window is at least one reporting cycle (~12 months).** HESA returns are annual, so an attribute can sit untouched for 10 months and then matter at return time. New attributes also get a grace period.
- **Flagging is not deleting.** A flag only goes to the approver. Even “retire” just hides the attribute from the picker, entry grid and export join; its values stay in the database and it can be restored. Nothing is removed automatically.

Known gap: exports don't yet record which fields each run used. A small usage log on the compile/export step would add a fifth signal (“last used in a submission on...”); the mapping check already covers most of it.

How it fits the current code

Piece	Change
Data model	student_custom_field gains status (pending / approved / rejected / retired), requested_by, approved_by, decided_at and the AI assessment as JSON. Additive migration.
Approval	A task in the existing workflow / task / notification engine, gated by a new permission code (permission-driven UI pattern).
AI checks	Existing app/ai classify / rank / read pattern. Model off → deterministic fallback: string similarity for duplicates, reason-to-spec-code matching for necessity. The gate still works without an API key.
Queries	Picker, entry grid and build_records filter to status = approved.
Privacy & tenancy	Only labels, reasons and spec text go to the model — never student values. Prompts contain only the current tenant's definitions (RLS-scoped).

Suggested phasing

Phase	Scope	Outcome
A	Layer 1 (duplicate check) + Layer 3 (approval)	No attribute is created without a second person; obvious duplicates blocked
B	Layer 2 (necessity check, type/code inference)	Every attribute is tied to a real return field and typed from day one
C	Layer 4 (review) + export usage log	Unused attributes are surfaced for retirement each cycle